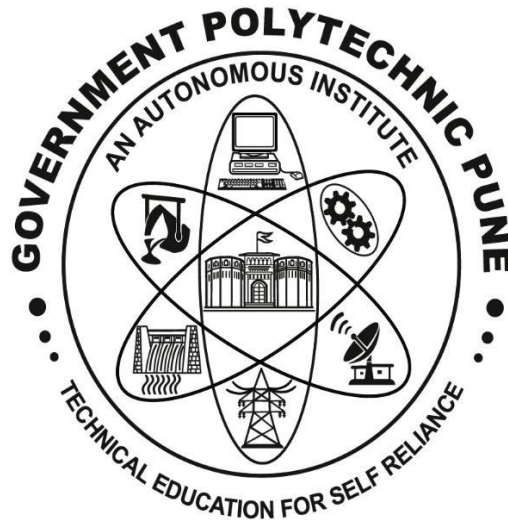


GOVERNMENT POLYTECHNIC PUNE
(An Autonomous Institute of Government of Maharashtra)
University Road, Shivajinagar, Pune-411016

DEPARTMENT OF COMPUTER ENGINEERING
Academic Year 2025-26



A Micro Project Report On
“Medicare – Cloud-Based Hospital Management System”

Submitted By

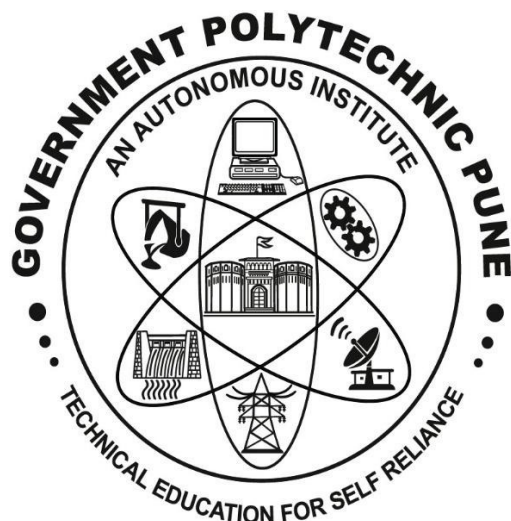
Pravin Kadam	2406088
Mohit Jagtap	2406080
Pranav Jagtap	2406082
Avadhoot Kamthe	2406094

Guided by:

Mrs.R.J.Chavan
Lecturer, Government Polytechnic Pune

GOVERNMENT POLYTECHNIC PUNE
(An Autonomous Institute of Government of Maharashtra)
University Road, Shivajinagar, Pune-411016

DEPARTMENT OF COMPUTER ENGINEERING
Academic Year 2025-26



CERTIFICATE

This is to certify that the micro-project work entitled **“Medicare – Cloud-Based Hospital Management System.”** is a bonafide work carried out by

Pravin Kadam	2406088
Mohit Jagtap	2406080
Pranav Jagtap	2406082
Avadhoot Kamthe	2406094

Of class Second Year in partial fulfilment of requirement for completion of course Cloud Computing (CM51207) of Diploma in Computer Engineering from Government Polytechnic, Pune. The report has been approved as it satisfies the academic requirements in respect of microproject work prescribed for the course.

Mrs.R.J.Chavan

(Microproject Guide)

Smt.J.R.Hange

(Head of Department)

Dr.R.K.Patil

(Principal)

ACKNOWLEDEMENT

I extend my deepest gratitude to **Dr. Rajendra K. Patil**, our esteemed Principal, for his unwavering support, encouragement, and motivation throughout the journey of my project, **“Medicare – Cloud-Based Hospital Management System.”** His vision and encouragement provided me with the inspiration to successfully complete this project.

Special recognition is due to **Mrs. J. R. Hange**, our dedicated Head of Department, for her invaluable guidance, insightful suggestions, and constant motivation, which played a significant role in shaping this project.

I express my heartfelt gratitude to **Mrs. R. J. Chavan**, my project mentor, whose patience, continuous support, and valuable advice guided me throughout the development of this project. Her expertise and constructive feedback helped me overcome challenges and gain a better understanding of cloud-based web applications, secure authentication, and healthcare data management.

Furthermore, I acknowledge that the successful completion of this project reflects my dedication, hard work, and perseverance. The knowledge and practical experience gained during this project have contributed significantly to my technical skills and understanding of cloud computing and web development.

In closing, I extend my sincere thanks to all those who contributed directly or indirectly to the successful completion of this project. Their support and encouragement have been invaluable throughout this journey.

ABSTRACT

Hospitals and clinics handle a large amount of information every day, such as patient records, doctor details, appointments, and prescriptions. Managing this information manually or with paper records is slow, error-prone, and difficult to access when needed.

The **Medicare – Cloud-Based Hospital Management System** is developed to connect patients, doctors, and administrators through one secure web platform. The system uses Supabase cloud services for authentication, PostgreSQL database, and storage, and is deployed on Vercel so that it can be accessed from any device with a browser.

The application allows patients to register, search doctors, book appointments using time slots, and view prescriptions. Doctors can manage appointments and create prescriptions, while administrators can add and manage doctors, patients, and appointments. Secure login is provided using mobile number, password, and SMS OTP verification.

This project helps bridge the gap between theoretical concepts of **cloud computing, databases, web development, and security** and their practical implementation. It provides a reliable and scalable approach to digital hospital management in real-world scenarios.

INDEX

Sr no	Content	Page
1.	Introduction	1
2.	Requirement Analysis	2
3.	Implementation	3-4
4.	Code	5-14
5.	Output	15-19
6.	Advantages	20
7.	Disadvantage	21
8.	Future Scope	22
9.	Course Level Outcomes Achieved	23
10.	Conclusion	24
11.	References	25

1.Introduction

Healthcare services depend on accurate and timely information. Hospitals have to manage patient details, doctor schedules, appointments, prescriptions, and medical history every day for a large number of people.

Handling such information manually causes long waiting times, lost records, and appointment conflicts.

Understanding concepts such as cloud computing, cloud databases, authentication, role-based access control, and web hosting is essential for computer engineering students. However, these concepts can be difficult to understand through theoretical study alone without building and deploying a real cloud-based application.

The **Medicare – Cloud-Based Hospital Management System** is developed to demonstrate how a hospital can manage its daily activities using cloud technologies. The system connects patients, doctors, and administrators through a responsive web application that uses Supabase as a cloud backend for authentication, data storage, and file storage.

The application provides features such as patient and doctor registration, secure login with SMS OTP, doctor search, appointment booking with time slots, appointment management, digital prescriptions, profile management, and an admin panel for managing doctors, patients, and appointments.

By combining cloud computing, web development, database management, and security concepts, this project helps users understand how real-world healthcare applications are built, deployed, and accessed from anywhere through the cloud.

2.Requirement Analysis

- A computer system with a stable internet connection is required to access and operate the cloud-based Medicare web application.
- A processor with minimum Intel Core i3 or equivalent capability is recommended for smooth application execution.
- At least 4 GB RAM is required to run the application, browser, and supporting services efficiently.
- Minimum 2 GB available storage space is recommended for project files, Node.js dependencies, and build output.
- A modern web browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox is required to access the application.
- Visual Studio Code or a similar code editor is required for developing and managing the project source code.
- Node.js and npm are required to run the Express server, the build script, and the OTP API endpoints.
- Supabase cloud services (Auth, PostgreSQL database, and Storage) are required to store users, doctors, patients, appointments, and prescriptions.
- A Twilio account is required to send SMS-based One-Time Passwords (OTP) for secure two-factor login verification.
- Vercel or a similar cloud hosting platform is required for deployment, and Row Level Security (RLS) must be enabled in Supabase.

3.Implementation Steps

The Medicare – Cloud-Based Hospital Management System is implemented as a responsive web application using HTML, CSS, JavaScript, Node.js (Express), and Supabase cloud services. The system allows patients, doctors, and administrators to log in to their own dashboards, manage appointments and prescriptions, and store all data securely in the cloud. The application is deployed on Vercel and can be accessed from any device through a web browser.

The implementation of the project includes the following modules:

- **User Registration and Authentication**

This module manages patient and doctor registration and login using mobile number and password. Supabase Auth is used to create user accounts, and each user profile is stored with a role such as patient, doctor, or admin.

- **Two-Factor OTP Verification**

After the password is verified, a 6-digit One-Time Password is generated on the server and sent to the registered mobile number using Twilio SMS. The OTP is stored as a salted SHA-256 hash, expires after a short time, and has limited attempts.

- **Role-Based Access Control**

Each page checks the role of the logged-in user. Patients, doctors, and administrators are redirected to their own dashboards and can only access the pages and data allowed for their role.

- **Cloud Database and Storage**

Profiles, patients, doctors, appointments, and prescriptions are stored in the Supabase PostgreSQL database. Profile photos are stored in Supabase Storage, and Row Level Security policies protect the data.

- **Patient Module**

This module allows patients to view their dashboard, search doctors by name or specialization, view doctor details, book appointments, track appointment status, view prescriptions, and update their profile.

- Appointment Booking and Slot Management

This is the main module of the system. The patient selects a doctor and a date, and the system shows morning and afternoon time slots. Slots that are already booked are shown as unavailable, which prevents double booking. The appointment is saved with a pending status until it is confirmed by the doctor.

- Doctor Module

Doctors can view their dashboard with appointment statistics, confirm, complete, or cancel appointments, view the list of their patients, and update their professional profile such as specialization, fees, and timings.

- Prescription Management

After an appointment is completed, the doctor can create a digital prescription containing medicine, dosage, frequency, duration, and instructions. Patients can view these prescriptions from their account.

- Admin Module

The administrator can view hospital statistics, add new doctors, activate or deactivate doctor profiles, and view all patients and appointments from a single admin dashboard.

- Deployment and System Integration

All modules are integrated into a single cloud-based application. A build script prepares the static files, the Express server provides the OTP APIs, and the project is deployed on Vercel with Supabase as the backend.

This implementation demonstrates practical concepts of cloud computing, cloud databases, authentication, role-based access control, and web deployment, providing a reliable approach for digital hospital management.

4. Code:

File 1: server.js – Express server with static hosting and OTP API routes

```
import express from 'express';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import fs from 'node:fs';
import { execSync } from 'node:child_process';
import { sendOtp, verifyOtp } from './server/otpService.js';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const distDir = path.join(__dirname, 'dist');

// Ensure dist exists; build static assets
try {
  console.log('[server] Building static site...');
  execSync('node build.js', { stdio: 'inherit', cwd: __dirname });
} catch (err) {
  console.error('[server] Build warning:', err.message);
}

const app = express();
const PORT = process.env.PORT || 3000;
const HOST = '0.0.0.0';

// JSON parsing for API requests
app.use(express.json());

// Twilio SMS OTP Endpoints
app.post('/api/auth/send-otp', async (req, res) => {
  try {
    const { phone, userId } = req.body || {};
    const result = await sendOtp(phone, userId);
    res.status(result.statusCode || (result.success ? 200 : 400)).json(result);
  } catch (err) {
    console.error('[API send-otp error]:', err.message);
    res.status(500).json({ success: false, error: 'Internal server error while sending OTP.' });
  }
});

app.post('/api/auth/verify-otp', async (req, res) => {
  try {
    const { phone, otp } = req.body || {};
    const result = await verifyOtp(phone, otp);
    res.status(result.statusCode || (result.success ? 200 : 400)).json(result);
  } catch (err) {
    console.error('[API verify-otp error]:', err.message);
    res.status(500).json({ success: false, error: 'Internal server error while verifying OTP.' });
  }
});
```

File 2: server/otpService.js – Secure OTP generation, hashing and SMS sending

```
// In-memory OTP storage
// Key: normalized phone (e.g. "9876543210")
// Value: { hashedOtp, salt, expiresAt, attemptsLeft, lastSentAt, requestCount, windowStart }
const otpStore = new Map();

// Helper to normalize phone number consistent with Medicare app
export function normalizePhone(raw) {
  let s = String(raw || "").trim().toLowerCase();
  if (s.startsWith('+91')) s = s.slice(3);
  else if (s.startsWith('+1')) s = s.slice(2);
  else if (s.startsWith('+')) s = s.slice(1);
  let d = s.replace(/\D/g, "");
  if (d.length === 12 && d.startsWith('91')) d = d.slice(2);
  if (d.length > 10 && d.startsWith('0')) d = d.slice(1);
  if (d.length === 11 && d.startsWith('1')) d = d.slice(1);
  return d;
}

// Convert normalized phone to E.164 for Twilio SMS
export function toE164(raw) {
  let cleaned = String(raw || "").trim();
  if (cleaned.startsWith('+')) {
    return '+' + cleaned.slice(1).replace(/\D/g, "");
  }
  let digits = cleaned.replace(/\D/g, "");
  if (digits.length === 10) {
    return '+91' + digits; // Default country code India (+91)
  }
  if (digits.length === 12 && digits.startsWith('91')) {
    return '+' + digits;
  }
  return '+' + digits;
}

// Cryptographically secure 6-digit OTP generation (100000 - 999999)
export function generateSecureOtp() {
  return crypto.randomInt(100000, 1000000).toString();
}

// Hash OTP with salt using SHA-256
export function hashOtp(otp, salt) {
  return crypto.createHash('sha256').update(`${salt}:${otp}`).digest('hex');
}

/**
 * Send OTP via Twilio SMS
 */
```

File 3: js/app.js – Supabase client, session handling and role-based layout

```
const {
  createClient
} = window.supabase;
const cfg = window.MEDICARE_CONFIG || {};
if (!cfg.url || !cfg.key) throw new Error('Missing Supabase configuration. Run the build with .env
  configured.');
```

```
export const supabase = createClient(cfg.url, cfg.key, {
  auth: {
    persistSession: true,
    autoRefreshToken: true,
    detectSessionInUrl: true
  }
});
```

```
export const $ = (s, r = document) => r.querySelector(s);
export const $$ = (s, r = document) => [...r.querySelectorAll(s)];
export const esc = s => String(s ?? "").replace(/[\&<>"])/g, c => ({
  '&': '&amp;',
  '<': '&lt;',
  '>': '&gt;',
  '"': '&#39;',
  "'": '&quot;'
})[c]);
```

```
export function toast(msg) {
  const e = $('#toast');
  if (e) {
    e.textContent = msg;
    e.classList.add('show');
    setTimeout(() => e.classList.remove('show'), 3000)
  }
}
```

```
export function normalizePhone(raw) {
  let s = String(raw || "").trim().toLowerCase();
  if (s.startsWith('+91')) s = s.slice(3);
  else if (s.startsWith('+1')) s = s.slice(2);
  else if (s.startsWith('+')) s = s.slice(1);
  let d = s.replace(/D/g, "");
  if (d.length === 12 && d.startsWith('91')) d = d.slice(2);
  if (d.length > 10 && d.startsWith('0')) d = d.slice(1);
  if (d.length === 11 && d.startsWith('1')) d = d.slice(1);
  return d
}
```

```
export const syntheticEmail = p => `${normalizePhone(p)}@medicare.local`;
export async function profile(uid) {
  const {
    data,
    error
```

File 4: build.js – Build script that prepares the deployable dist folder

```
import fs from 'node:fs';
import path from 'node:path';
const root = process.cwd();

function loadEnv() {
  const p = path.join(root, '.env');
  if (!fs.existsSync(p)) return {};
  const o = {};
  for (const line of fs.readFileSync(p, 'utf8').split(/\r?\n/)) {
    const m = line.match(/^(?[A-Za-z_][\w]*)=?(?<value>.*$)/);
    if (m) o[m[1]] = m[2].replace(/^["']|["']$/g, '');
  }
  return o;
}

const env = {
  ...loadEnv(),
  ...process.env
};

const url = env.SUPABASE_URL || env.NEXT_PUBLIC_SUPABASE_URL ||
  'https://tbzjajeaknmflsboiwki.supabase.co';
const key = env.SUPABASE_ANON_KEY || env.SUPABASE_PUBLISHABLE_KEY ||
  env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY ||
  'sb_publishable_ujRYPG9FW9lFKKN7vbBcLQ_gUHfSdmG';
if (!url || !key || /your[-_]/i.test(url) || /your[-_]/i.test(key)) throw new Error('SUPABASE_URL
  and SUPABASE_ANON_KEY (or publishable key) must be set in .env');
if (!/^https:\/\/[A-Za-z0-9.-]+\supabase\.co\/?$/.test(url)) throw new Error('Invalid
  SUPABASE_URL');
if (/service[-_]?role|secret/i.test(key)) throw new Error('A service-role/secret key cannot be used
  in the browser');

const dist = path.join(root, 'dist');
fs.rmSync(dist, {
  recursive: true,
  force: true
});
fs.mkdirSync(dist, {
  recursive: true
});

function cp(src, dst = src) {
  const s = path.join(root, src),
    d = path.join(dist, dst);
  if (fs.existsSync(s)) {
    fs.mkdirSync(path.dirname(d), {
      recursive: true
    });
    fs.cpSync(s, d, {
      recursive: true
    });
  }
}
```

File 5: login.html – Login with mobile number, password and OTP verification

```
<script type="module">
import {
  supabase,
  normalizePhone,
  syntheticEmail,
  $
} from './js/app.js';

const credentialsView = $('#credentialsView');
const otpView = $('#otpView');
const successView = $('#successView');

const loginForm = $('#loginForm');
const msg = $('#msg');
const loginBtn = $('#loginSubmitBtn');

const otpForm = $('#otpForm');
const otpMsg = $('#otpMsg');
const verifyBtn = $('#verifyOtpBtn');
const resendBtn = $('#resendOtpBtn');
const timerText = $('#timerText');
const countdownSecs = $('#countdownSecs');
const maskedPhoneDisplay = $('#maskedPhoneDisplay');
const changeNumberBtn = $('#changeNumberBtn');

const otpInputs = [...document.querySelectorAll('.otp-input')];

let currentPhone = "";
let verifiedUser = null;
let countdownInterval = null;

// OTP Digit navigation handlers
otpInputs.forEach((input, index) => {
  input.addEventListener('input', (e) => {
    const val = input.value.replace(/D/g, "");
    input.value = val ? val[val.length - 1] : "";
    if (input.value && index < otpInputs.length - 1) {
      otpInputs[index + 1].focus();
    }
  });

  input.addEventListener('keydown', (e) => {
    if (e.key === 'Backspace') {
      if (!input.value && index > 0) {
        otpInputs[index - 1].focus();
        otpInputs[index - 1].value = "";
      }
    }
  });
});
```

File 6: register.html – Patient / doctor registration using Supabase Auth

```
<script type="module">
import {
  supabase,
  normalizePhone,
  syntheticEmail,
  $,
  toast
} from './js/app.js';
const f = $('#regForm'),
  m = $('#msg');
const role = f.querySelector('[name=role]');

function toggle() {
  document.querySelectorAll('.doctorOnly').forEach(x => x.style.display = role.value === 'doctor' ?
    'block' : 'none')
}
role.addEventListener('change', toggle);
toggle();
f.addEventListener('submit', async e => {
  e.preventDefault();
  m.innerHTML = "";
  const fd = new FormData(f);
  const name = fd.get('name').trim(),
    phone = normalizePhone(fd.get('phone')),
    pass = fd.get('password');
  if (pass !== fd.get('confirm')) {
    m.innerHTML = '<div class="alert error">Passwords do not match.</div>';
    return
  }
  if (pass.length < 6 || phone.length < 10) {
    m.innerHTML = '<div class="alert error">Use a valid mobile number and password of at least 6
      characters.</div>';
    return
  }
  const r = fd.get('role');
  const extra = {
    date_of_birth: fd.get('dob') || '1995-01-01',
    dob: fd.get('dob') || '1995-01-01',
    gender: fd.get('gender'),
    blood_group: fd.get('blood'),
    address: fd.get('address'),
    emergency_contact: fd.get('emergency'),
    medical_history: fd.get('history'),
    specialization: fd.get('specialization'),
    qualification: fd.get('qualification')
  };
};
```

File 7: patient/book.html – Appointment booking with time-slot selection

```
<script type="module">
import {
  requireRole,
  supabase,
  shell,
  $,
  $$,
  esc,
  roleRow
} from '../js/app.js';

const p = await requireRole('patient');
if (p) {
  const row = await roleRow('patient', p.id);
  const did = new URLSearchParams(location.search).get('doctor');
  const {
    data: docs = []
  } = await supabase.from('doctors').select('*', profiles('*')).eq('available', true);

  // Local calendar date helper avoiding timezone shift
  const dNow = new Date();
  const todayStr = `${dNow.getFullYear()}-${String(dNow.getMonth() + 1).padStart(2,
    '0')}-${String(dNow.getDate()).padStart(2, '0')}`;

  const MORNING_SLOTS = [
    '10:00 AM',
    '10:30 AM',
    '11:00 AM',
    '11:30 AM',
    '12:00 PM',
    '12:30 PM'
  ];

  const AFTERNOON_SLOTS = [
    '2:00 PM',
    '2:30 PM',
    '3:00 PM',
    '3:30 PM',
    '4:00 PM',
    '4:30 PM',
    '5:00 PM'
  ];

  function normTime(t) {
    if (!t) return "";
    return String(t)
  }
}
```

File 8: patient/dashboard.html – Patient dashboard with appointments and prescriptions

```

<script type="module">
import {
  requireRole,
  roleRow,
  supabase,
  shell,
  $,
  esc,
  status
} from './js/app.js';
const p = await requireRole('patient');
if (p) {
  const row = await roleRow('patient', p.id);
  const {
    data: apps = []
  } = await supabase.from('appointments').select('*').eq('patient_id',
    row.id).order('appointment_date', {
      ascending: true
    });
  const {
    data: rx = []
  } = await supabase.from('prescriptions').select('*').eq('patient_id', row.id).order('created_at',
    {
      ascending: false
    }).limit(3);
  shell('patient', 'dashboard.html', 'Dashboard', `<div class="page-head"><div><h1>Welcome,
    ${esc(p.name)}</h1><p class="muted">Your healthcare overview</p></div></div><div
    class="stats"><div class="card stat"><b>${apps.length}</b><span>Total
    appointments</span></div><div class="card
    stat"><b>${apps.filter(x=>x.status==='pending').length}</b><span>Pending</span></div><div
    class="card
    stat"><b>${apps.filter(x=>x.status==='confirmed').length}</b><span>Confirmed</span></div><div
    class="card stat"><b>${rx.length}</b><span>Recent prescriptions</span></div></div><section
    style="margin-top:22px" class="card"><h2>Upcoming appointments</h2>${apps.length?'<div
    class="table-wrap"><table
    class="table"><tr><th>Date</th><th>Time</th><th>Reason</th><th>Status</th></tr>${apps.slice(0,
    5).map(a=>`<tr><td>${esc(a.appointment_date)}</td><td>${esc(a.appointment_time)}</td><td>${esc
    (a.reason||'-')}</td><td>${status(a.status)}</td></tr>`).join("")}</table></div>':<div
    class="empty">No appointments yet. <a href="doctors.html">Find a
    doctor</a>.</div>'</section>`)
  }
</script>

```

File 9: doctor/prescription-new.html – Doctor prescription creation module

```

<script type="module">
import {
  requireRole,
  supabase,
  shell,
  roleRow,
  esc,
  $
} from './js/app.js';
const p = await requireRole('doctor');
if (p) {
  const d = await roleRow('doctor', p.id);
  const {
    data: a = []
  } = await supabase.from('appointments').select('*',patients(*,profiles(*))).eq('doctor_id',
    d.id).eq('status', 'completed');
  const id = new URLSearchParams(location.search).get('appointment');
  shell('doctor', 'prescription-new.html', 'Prescription', `
<div class="prescription-shell">
  <div class="prescription-hero"><div class="prescription-icon">Rx</div><div><h1>New
    prescription</h1><p>Create a clear prescription for a completed patient
    appointment.</p></div></div>
  <div id="msg"></div>
  <form id="rx" class="card prescription-form-card">
    <div class="appointment-select-wrap"><div class="field"><label>Completed
      appointment</label><select name="appointment" required>${a.length?a.map(x=>`<option
      value="${x.id}" ${x.id===id?selected:'"}>${esc(x.patients?.profiles?.name||'Patient')}` —
      ${esc(x.appointment_date)}${x.appointment_time?' '
      '+esc(x.appointment_time):"")}</option>`)}.join("):'<option value="">No completed
      appointments available</option>'}</select><span class="field-hint">Only completed
      appointments can receive a prescription.</span></div></div>
    <div class="form-grid" style="margin-top:20px">
      <div class="field"><label>Medicine</label><input name="medicine" required placeholder="e.g.
        Paracetamol 500 mg"></div>
      <div class="field"><label>Dosage</label><input name="dosage" required placeholder="e.g. 1
        tablet"></div>
      <div class="field"><label>Frequency</label><input name="frequency" required
        placeholder="e.g. Twice daily"></div>
      <div class="field"><label>Duration</label><input name="duration" placeholder="Optional —
        e.g. 5 days"></div>
      <div class="field" style="grid-column:1/-1"><label>Instructions</label><textarea
        name="instructions" placeholder="Add instructions for the patient..."></div>
    </div>
    <div class="prescription-actions"><button type="button" class="btn btn-ghost"
      onclick="history.back()">Cancel</button><button class="btn btn-primary">Save
      prescription</button></div>
  `

```

File 10: admin/add-doctor.html – Admin module for adding a new doctor

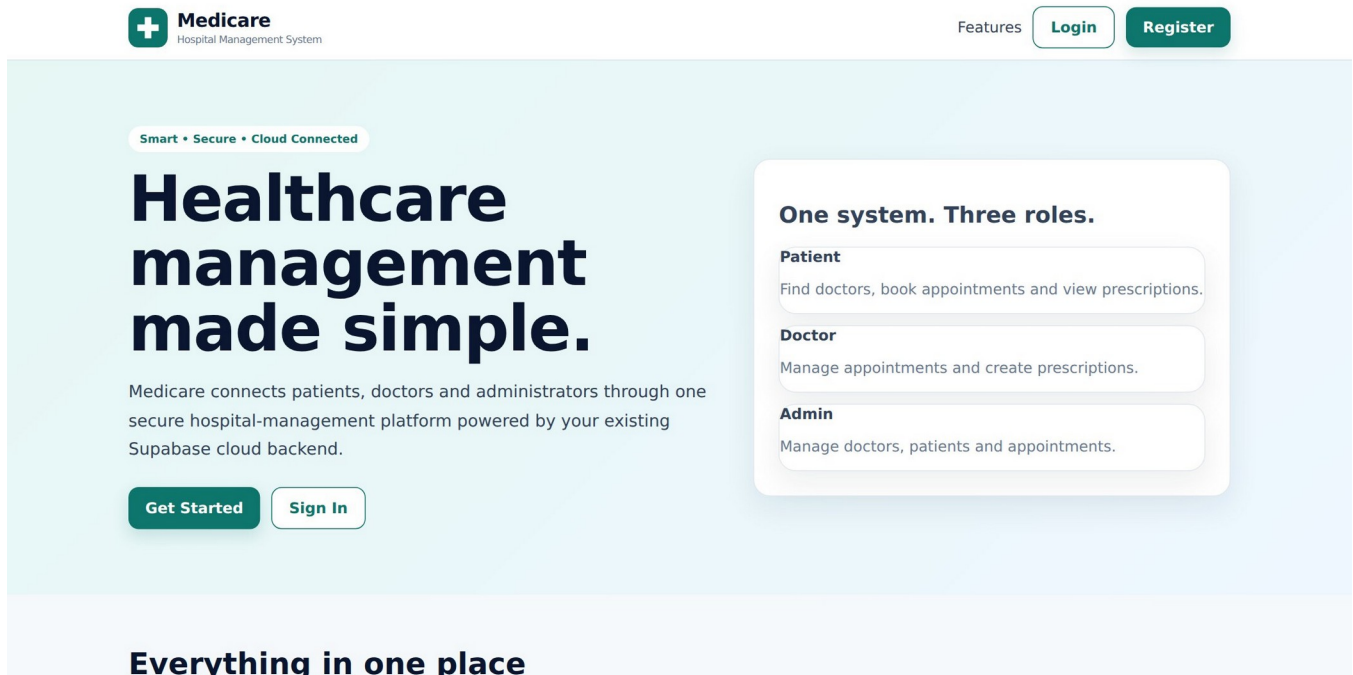
```

<script type="module">
import {
  requireRole,
  supabase,
  shell,
  $,
  esc,
  normalizePhone,
  syntheticEmail
} from './js/app.js';
const p = await requireRole('admin');
if (p) {
  shell('admin', 'add-doctor.html', 'Add Doctor', `<div class="page-head"><div><h1>Add
  doctor</h1><p class="muted">Create a doctor account using Supabase Auth.</p></div></div><div
  class="card"><div id="msg"></div><form id="add"><div class="form-grid"><div
  class="field"><label>Name</label><input name="name" required></div><div
  class="field"><label>Mobile</label><input name="phone" required></div><div
  class="field"><label>Password</label><input name="password" type="password" minlength="6"
  required></div><div class="field"><label>Specialization</label><input name="spec"
  value="General Medicine"></div><div class="field"><label>Qualification</label><input
  name="qual" value="MBBS, MD"></div><div class="field"><label>Experience</label><input
  name="exp" value="5+ Years"></div><div class="field"><label>Hospital</label><input
  name="hospital" value="Medicare Central Hospital"></div><div><div
  class="field"><label>Department</label><input name="dept" value="General Medicine"></div><div
  class="field"><label>Consultation fee</label><input name="fee" type="number"
  value="500"></div><div class="field"><label>Available days</label><input name="days"
  value="Mon,Tue,Wed,Thu,Fri"></div><div class="field"><label>Start time</label><input
  name="start" value="09:00 AM"></div><div class="field"><label>End time</label><input
  name="end" value="05:00 PM"></div><div class="field"
  style="grid-column:1/-1"><label>Bio</label><textarea name="bio">Experienced
  clinician.</textarea></div></div><button class="btn btn-primary">Create
  doctor</button></form></div>`);
  $('#add').onsubmit = async e => {
    e.preventDefault();
    const f = new FormData(e.currentTarget),
      phone = normalizePhone(f.get('phone'));
    const isolated = window.supabase.createClient(window.MEDICARE_CONFIG.url,
      window.MEDICARE_CONFIG.key, {
      auth: {
        persistSession: false,
        autoRefreshToken: false
      }
    });
    try {
      const {
        data,

```

5. Output:

HOME PAGE



The home page features a light blue header with the Medicare logo (a green plus sign) and the text "Medicare Hospital Management System". To the right of the logo are links for "Features", "Login", and "Register". The main content area has a light blue background. On the left, there's a large heading "Healthcare management made simple." followed by a subtext: "Medicare connects patients, doctors and administrators through one secure hospital-management platform powered by your existing Supabase cloud backend." Below this are two buttons: "Get Started" and "Sign In". On the right, there's a white box titled "One system. Three roles." containing three sections: "Patient" (Find doctors, book appointments and view prescriptions.), "Doctor" (Manage appointments and create prescriptions.), and "Admin" (Manage doctors, patients and appointments.). At the bottom, a dark blue bar contains the text "Everything in one place".

Medicare
Hospital Management System

Features [Login](#) [Register](#)

Smart • Secure • Cloud Connected

Healthcare management made simple.

Medicare connects patients, doctors and administrators through one secure hospital-management platform powered by your existing Supabase cloud backend.

[Get Started](#) [Sign In](#)

One system. Three roles.

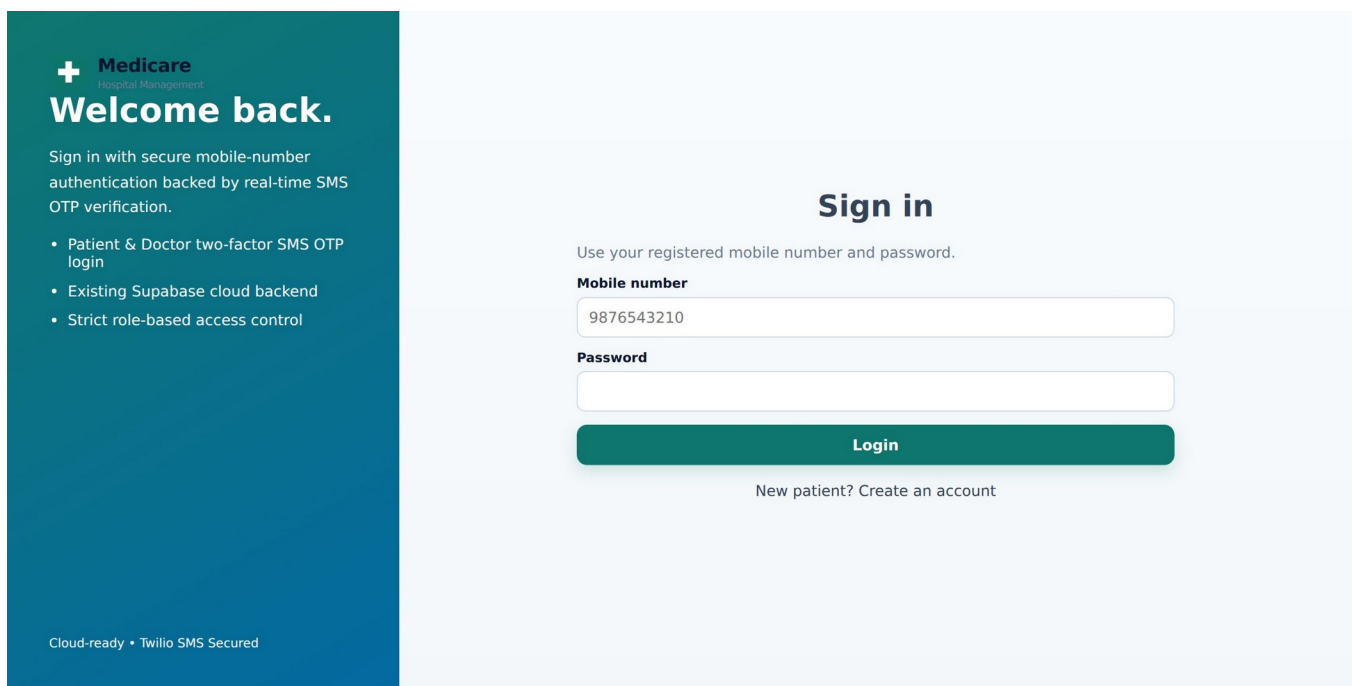
Patient
Find doctors, book appointments and view prescriptions.

Doctor
Manage appointments and create prescriptions.

Admin
Manage doctors, patients and appointments.

Everything in one place

LOGIN PAGE



The login page has a dark blue sidebar on the left with the Medicare logo and the text "Welcome back." followed by a description: "Sign in with secure mobile-number authentication backed by real-time SMS OTP verification." Below this is a list of features: "Patient & Doctor two-factor SMS OTP login", "Existing Supabase cloud backend", and "Strict role-based access control". At the bottom of the sidebar, it says "Cloud-ready • Twilio SMS Secured". The main content area has a light blue background. It features a heading "Sign in" followed by the text "Use your registered mobile number and password." Below this are two input fields: "Mobile number" (containing "9876543210") and "Password". A large green "Login" button is positioned below the input fields. At the bottom, there's a link: "New patient? Create an account".

Medicare
Hospital Management

Welcome back.

Sign in with secure mobile-number authentication backed by real-time SMS OTP verification.

- Patient & Doctor two-factor SMS OTP login
- Existing Supabase cloud backend
- Strict role-based access control

Cloud-ready • Twilio SMS Secured

Sign in

Use your registered mobile number and password.

Mobile number

Password

[Login](#)

[New patient? Create an account](#)

PATIENT DASHBOARD



PATIENT MENU

Dashboard

Doctors

Book Appointment

My Appointments

Prescriptions

My Profile

Logout

Welcome, Pravin Kadam

Your healthcare overview

3

Total appointments

0

Pending

2

Confirmed

3

Recent prescriptions

Upcoming appointments

Date	Time	Reason	Status
2026-10-06	10:30 AM	Chest pain follow-up	Confirmed
2026-10-14	12:00 PM	Child vaccination	Cancelled
2026-10-09	11:30 AM	Headache	Confirmed

FIND A DOCTOR



PATIENT MENU

Dashboard

Doctors

Book Appointment

My Appointments

Prescriptions

My Profile

Logout

Find a doctor

Browse available doctors.

Search by doctor name o

All specializations ▾



Dr. Anjali Deshmukh

Cardiology

MBBS, MD (Cardiology)

8+ Years · Medicare Central Hospital

View doctor profile



Dr. Rahul Patil

General Medicine

MBBS, MD

11+ Years · Medicare Central Hospital

View doctor profile



Dr. Sneha Kulkarni

Pediatrics

MBBS, DCH

14+ Years · Medicare Central Hospital

View doctor profile



Dr. Amit Joshi

Orthopedics

BOOK APPOINTMENT



PATIENT MENU

Dashboard

Doctors

Book Appointment

My Appointments

Prescriptions

My Profile

Logout

Book appointment

Select a doctor and date, then choose an available time slot.

Doctor

Dr. Anjali Deshmukh — Cardiology

Appointment Date

10/12/2026

Reason for Visit

General consultation

Available Appointment Slots

Click any **Available** slot to book your appointment immediately.

MORNING SESSIONS (10:00 AM - 1:00 PM)

10:00 AM Available	10:30 AM Available	11:00 AM Available	11:30 AM Available	12:00 PM Available	12:30 PM Available
-----------------------	-----------------------	-----------------------	-----------------------	-----------------------	-----------------------

----- Lunch Break: 1:00 PM - 2:00 PM -----

AFTERNOON SESSIONS (2:00 PM - 5:00 PM)

PATIENT PRESCRIPTIONS



PATIENT MENU

Dashboard

Doctors

Book Appointment

My Appointments

Prescriptions

My Profile

Logout

Prescriptions

Your prescriptions from doctors.

Atorvastatin 10mg

Dosage: 1 tablet

Frequency: Once daily (night)

Instructions: Take after dinner for 30 days

Dr. Dr. Anjali Deshmukh

Paracetamol 500mg

Dosage: 1 tablet

Frequency: Twice daily

Instructions: After meals for 5 days

Dr. Dr. Rahul Patil

Vitamin D3 60K

Dosage: 1 capsule

Frequency: Once weekly

Instructions: With milk for 8 weeks

Dr. Dr. Sneha Kulkarni

Department of Computer Engineering , Government Polytechnic Pune

17

DOCTOR DASHBOARD



DOCTOR MENU

Dashboard

Appointments

Patients

Prescriptions

My Profile

Logout

Doctor Dashboard

Welcome back, Dr. Dr. Anjali Deshmukh. Here is today's overview.

3

Total appointments

0

Pending

1

Confirmed

1

Completed

Recent appointments

Your latest patient consultations.

Patient	Date	Time	Status
Pravin Kadam	2026-10-06	10:30 AM	Confirmed
Pravin Kadam	2026-10-14	12:00 PM	Cancelled
Mohit Jagtap	2026-09-28	2:00 PM	Completed

Add instructions for the patient...

NEW PRESCRIPTION

ADMIN DASHBOARD



ADMIN MENU

Dashboard

Manage Doctors

Add Doctor

Patients

Appointments

Logout

Admin Dashboard

Hospital operations overview

4
Doctors

4
Patients

8
Appointments

1
Pending

Appointment status

Pending: 1 · Confirmed: 3 · Completed: 3 · Cancelled: 1

MANAGE DOCTORS



ADMIN MENU

Dashboard

Manage Doctors

Add Doctor

Patients

Appointments

Logout

Manage doctors

View and deactivate doctor profiles.

Add doctor

Name	Specialization	Experience	Hospital	Status	
Dr. Anjali Deshmukh	Cardiology	8+ Years	Medicare Central Hospital	Available	Deactivate
Dr. Rahul Patil	General Medicine	11+ Years	Medicare Central Hospital	Available	Deactivate
Dr. Sneha Kulkarni	Pediatrics	14+ Years	Medicare Central Hospital	Available	Deactivate
Dr. Amit Joshi	Orthopedics	17+ Years	Medicare Central Hospital	Available	Deactivate

6. Advantages

1. **Centralized Cloud Data:**

All patient, doctor, appointment, and prescription records are stored in one secure cloud database.

2. **Access from Anywhere:**

The web application can be used from any computer or mobile browser without installing extra software.

3. **Easy Appointment Booking:**

Patients can book appointments online by choosing a doctor, date, and available time slot.

4. **Digital Prescriptions:**

Doctors create prescriptions digitally, so patients can view them anytime and paper records are reduced.

5. **Secure Authentication:**

Password login with SMS OTP verification and role-based access protects sensitive medical data.

6. **No Double Booking:**

Already booked time slots are shown as unavailable, which avoids appointment conflicts.

7. **Separate Role Dashboards:**

Patients, doctors, and administrators each get a simple dashboard designed for their own work.

8. **Reliable and Scalable:**

Cloud services like Supabase and Vercel make the system suitable for growing numbers of users and records.

7. Disadvantages

1. **Internet Dependency:**
The system requires a stable internet connection to log in, book appointments, and access records.
2. **Cloud Service Dependency:**
The system depends on the availability of Supabase, Vercel, and Twilio cloud services.
3. **Security Risks:**
Wrong configuration of database policies or API keys may expose sensitive patient information.
4. **SMS Cost and Delivery:**
Sending OTP messages through Twilio has a cost, and SMS delivery may be delayed in poor network areas.
5. **Limited Features:**
The system does not yet support online payments, video consultation, or detailed laboratory reports.
6. **Maintenance Requirements:**
Regular maintenance, security updates, database backups, and service monitoring are required.
7. **Privacy Concerns:**
Medical data is highly sensitive, so strict access control and secure cloud configuration are necessary.

8. Future Scope

1. **Online Payment Integration:**

Payment gateways can be added so patients can pay consultation fees online while booking appointments.

2. **Video Consultation:**

Telemedicine features can be added to allow patients to consult doctors through secure video calls.

3. **Electronic Health Records:**

Complete medical history, lab reports, and scans can be stored and shared securely between doctors.

4. **AI-Based Assistance:**

AI can be used for symptom checking, doctor recommendations, and predicting appointment demand.

5. **Notifications and Reminders:**

Automatic SMS, email, and push reminders can be sent for appointments and medicine schedules.

6. **Mobile Application Sync:**

The web system can be fully synchronized with the Medicare Android app using the same cloud backend.

7. **Analytics and Reports:**

Administrators can get detailed reports on appointments, doctor performance, and hospital usage.

8. **Multi-Hospital Support:**

The system can be extended to manage multiple hospitals, branches, and departments from one platform.

9. Course Level Outcomes Achieved:

1. Understand Cloud Computing Concepts

The project demonstrates important cloud computing concepts by using cloud services for authentication, database, storage, and hosting. It helps understand how applications are delivered over the internet.

2. Use Cloud Service Models

Supabase is used as a Backend-as-a-Service (BaaS) and Vercel as a Platform-as-a-Service (PaaS). This shows the practical use of cloud service models in a real application.

3. Implement Cloud Database and Storage

The system stores patient, doctor, appointment, and prescription data in a cloud PostgreSQL database and profile images in cloud storage for centralized data management.

4. Deploy Applications on the Cloud

The application is built and deployed on Vercel with environment variables and security headers. This demonstrates how web applications are deployed and scaled on cloud platforms.

5. Apply Cloud Security and Access Control

The project uses Supabase Auth, SMS OTP verification, role-based access, and Row Level Security. This helps understand how cloud applications protect sensitive data.

10. Conclusion:

The **Medicare – Cloud-Based Hospital Management System** was successfully developed to manage patients, doctors, appointments, and prescriptions using cloud-based technologies.

The project demonstrates how a hospital can move from paper records to a secure **cloud-based system** that can be accessed from anywhere through a web browser.

The system provides features such as patient and doctor registration, secure login with SMS OTP, doctor search, appointment booking with time slots, digital prescriptions, and an admin dashboard. The implementation helps in understanding the practical use of cloud computing, databases, and web development.

The appointment booking module shows available time slots and prevents double booking, while role-based access control ensures that each user can only see and manage the data allowed for their role.

Overall, this project helps bridge the gap between theoretical cloud computing concepts and their practical implementation, providing valuable knowledge about cloud databases, authentication, deployment, and healthcare data management.

11. References:

- **Rajkumar Buyya**, *Mastering Cloud Computing*, McGraw Hill Education.
- **Thomas Erl**, *Cloud Computing: Concepts, Technology & Architecture*, Pearson Education.
- **NIST**, *The NIST Definition of Cloud Computing* (Special Publication 800-145).
- **Supabase Documentation**, *Auth, Database, Storage and Row Level Security*, supabase.com/docs.
- **Vercel Documentation**, *Deploying and hosting web applications*, vercel.com/docs.
- **Node.js and Express Documentation**, *Server-side JavaScript and API development*.
- **Twilio Documentation**, *Programmable SMS for OTP verification*, twilio.com/docs.
- **MDN Web Docs**, *References for HTML, CSS, JavaScript, and Web APIs*.
- **Hospital Management System Study Materials**, *used for understanding patient, doctor, appointment, and prescription workflows*.